COLLEGE EVENTS MANAGEMENT SYSTEM

Cornerstone project work submitted to Aditya University in partial fulfilment of the requirements for the award of the Degree of

B.Tech. degree

in

Artificial Intelligence and Machine Learning (2026-2027)

SUBMITTED BY

B.LASYA SHARON	25B11AI098
ANANYA DAS	25B11AI037
K.KURMANADH	25B11AI614
SANANDA CH MEDHA	25B11AIA67

UNDER THE GUIDANCE OF

T.VIJAYALAKSHMI,

Assistant Professor,

Department of AIML



Department of Arificial Intelligence & Machine Learning
Aditya University, Aditya Nagar, ADB Road,
Surampalem, Kakinada, District, Andhra Pradesh, India – 533437

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

ADITYA UNIVERSITY, ADITYA NAGAR, SURAMPALEM.

CERTIFICATE

This is to certify that the OOPS Through C++ Cornerstone Project report entitled "**COLLEGE EVENTS MANAGEMENT SYSTEM**" is being submitted by B.Lasya Sharon (25B11AI098), Ananya Das(25B11037), K.Kurmanadh (25B11AI614) and Sananda Chowdhury Medha (25B11AIA67) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Cornerstone Project** for the course **OOPS Through C++ (2501CS08)** during the academic year **2026–2027**, and records the Source Code implementation with coding standard practices and documentation undertaken by the team.

.

Project Guide
T.Vijayalakshmi,
Assistant professor

Head of the Department
Dr. K. Naga Bhargavi,
Associate Professor

DECLARATION

We hereby declare that the OOPS Through C++ Cornerstone Project entitled "**COLLEGE EVENTS MANAGEMENT SYSTEM**" is a genuine work carried out by us. The Source Code implementation with coding standard practices and documentation presented in this report are our own work, completed under the guidance of **T.VIJAYALAKSHMI ,**Assistant Professor, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

B.Lasya Sharon 25b11AI098

Ananya Das(25B11037)

K.Kurmanadh (25B11AI614)

Sananda Chowdhury Medha(25B11AIA67)

Date: 16-10-2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide **Dr. T.VIJAYALAKSHMI**,Assistant Professor, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to **Dr. K Naga Bhargavi**, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We also extend our sincere thanks to **Dr. M V RAJESH,** Associate Dean, School of Computing, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh – Registrar, Dr. A. Ramesh - Pro VC (E&S), Dr. S. Rama sree - Pro VC (Academics), Dr. M B Srinivas – Vice Chancellor and Dr. M. Sreenivasa Reddy - Dy. Pro-Chancellor and Management, Aditya university**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank our **Teaching and Non-Teaching Staff, Parents and Friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Project Associates

B.Lasya Sharon (25B11AI098)

Ananya Das(25B11037)

K.Kurmanadh (25B11AI614)

Sananda Chowdhury Medha (25B11AIA67)

ABSTRACT

The College Events Management System is a software application designed to simplify the planning and management of college events. It allows organisers to create events, enter dates and venues, set participant capacity, and make event information available to students.

The system models the main entities of a college event environment using classes such as Event, Student, Registration and EventManager. Each class has a clear responsibility, helping to organise event information and registration records.

The project demonstrates object-oriented programming concepts including classes, objects, encapsulation, constructors, inheritance where appropriate, and reusable methods. Input validation prevents invalid capacity values and duplicate registrations.

The C++ prototype can be compiled with a C++17-compatible compiler. It can be tested using scenarios for adding events, displaying events, registering students, searching by event ID, rejecting invalid capacity and generating registration reports.

Keywords: College Events Management System, C++, Object-Oriented Programming, Inheritance, Polymorphism, Encapsulation, Exception Handling.

ABBREVIATIONS

Abbreviation	Full Form
OOP	Object-Oriented Programming
CEMS	College Events Management System
ID	Identifier
UI	User Interface
DBMS	Database Management System
SQL	Structured Query Language
C++	Programming Language
ER	Entity Relationship
CRUD	Create, Read, Update, Delete
I/O	Input / Output
API	Application Programming Interface
IDE	Integrated Development Environment
CSE	Computer Science & Engineering

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Fig 1.1	System architecture of the College Events Management System	3
Fig 1.2	Flowchart of the event management process	5
Fig 1.3	Class diagram of the College Events Management System	6
Fig 2.1	Event creation and event listing	14
Fig 2.2	Student registration and confirmation	15
Fig 2.3	Search and validation messages	15
Fig 2.4	Error handling for invalid operations	16
Fig 2.5	Change Student ID and exit	16
Fig 2.6	Event locking after three wrong Student ID attempts	17

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 1.1	Planned modules of the College Events Management System	2
Table 1.2	Software and hardware requirements	3
Table 1.3	Event categories and registration rules	6
Table 2.1	OOP concepts and their use in the project	8
Table 2.2	Test cases and results	17

TABLE OF CONTENTS

CONTENTS	**PAGE NO**
Title Page	I
Declaration	II
Certificate	III
Acknowledgement	IV
Abstract	V
Abbreviations	VI
List of Figures	VII
List of Tables	VIII
CHAPTER-1 **Project Planning**	1
1.1. Problem Statement	1
1.2. Project Objective	1
1.3. Planned Modules	2
1.4. System Architecture (Algorithm & Flowchart)	3
CHAPTER-2 **Project Implementation**	7
2.1. OOP Concepts Used for Implementation	7
2.2. Code	8
2.3. Output Screens	14
CHAPTER-3 **Conclusion & Future Scope**	19
3.1. Conclusion	19
3.2. Future Scope	19
References	20

CHAPTER 1

PROJECT PLANNING

1.1 Problem Statement

Colleges conduct technical, cultural, sports and academic events throughout the year. Managing event details and student registrations manually can lead to duplicate entries, missing information, confusion about event capacity and difficulty preparing participant lists.

The problem addressed in this project is to design and develop a College Events Management System that stores event details in an organised form and supports event creation, event listing, student registration, searching and registration reporting.

A manual or unstructured approach makes it difficult to update event information and verify registrations. A modular object-oriented design separates event data, student details and registration operations, making the system easier to understand, test and extend.

This is to certify that the cornerstone project entitled "COLLEGE EVENTS MANAGEMENT SYSTEM" is submitted by the students listed below in partial fulfilment for the award of the B.Tech Degree from Aditya University. It is a record of project work carried out under the supervision of the project guide.

1.2 Objective

The main objective is to build a console-based College Events Management System in C++ that helps organisers manage events and student registrations. The specific objectives are:

- To represent events, students and registrations using suitable classes and objects.
- To allow an organiser to add events with a title, category, date, venue and participant capacity.
- To display and search event details and allow students to register for available events.
- To apply encapsulation and modular design to keep event and registration data organised.
- To validate inputs, prevent duplicate registrations and reject registration when event capacity is full.
- To provide a simple, extensible design that can later support databases, user accounts and online access.

1.3 Planned Modules

The project is divided into eight modules. Each module is implemented by one or more classes, which keeps responsibilities separate and makes testing easier. Table 1.1 lists the modules and the classes that implement them.

The project is divided into eight modules. Each module is implemented by one or more classes, which keeps responsibilities separate and makes testing easier. Table 1.1 lists the modules and the classes that implement them.

Table 1.1 Planned modules of the College Events Management System

	Module	Classes involved	Description
	Event Creation	EventManager, Event	Allows the organiser to add an event with title, category, date, venue and capacity.
	Event Listing	EventManager, Event	Displays available events and their details.
	Student Registration	EventManager, Registration	Registers a student for a selected event and records the details.
	Capacity Validation	Event, EventManager	Prevents registration when the event reaches maximum capacity.
	Event Search	EventManager, Event	Finds an event using its ID or title.
	Registration List	Registration, EventManager	Displays the participants registered for a selected event.
	Input Validation	EventManager	Checks required fields and rejects invalid values.

	Module	Classes involved	Description
	Summary and Exit	EventManager	Returns control safely and displays a completion message.

The software and hardware needed to build and run the simulator are modest, as shown in Table 1.2.

Table 1.2 Software and hardware requirements

Component	Specification
Programming language	C++ (ISO C++17 or later)
Operating system	Windows 10/11, Linux or macOS
Development tools	VS Code, Code::Blocks or any C++ IDE
Hardware	Intel Core i3 or above, 4 GB RAM, basic storage

1.4 System Architecture (Algorithm & Flowchart)

1.4.1 System architecture

The system follows a simple layered architecture. The presentation layer accepts menu choices and displays messages. The application layer handles event creation, search and registration rules. The data layer stores event and registration objects during program execution.

Fig 1.1 System architecture of the College Events Management System

1.4.2 Algorithm

The overall working of the College Events Management System is described by the following algorithm.

1. Start the program and initialise the event manager.
2. Display the main menu with options to add, view, search, register and report.
3. Read the organiser or student menu choice and validate the input.

4. If Add Event is selected, read event details and verify that capacity is greater than zero.
5. If View Events is selected, display the saved event list.
6. If Search Event is selected, find an event using its ID or title.
7. We express our sincere gratitude to our Cornerstone Project Guide and the faculty members who provided guidance, support, encouragement and valuable suggestions throughout the development of the College Events Management System.
8. If Register Student is selected, read the event ID and student details and check whether the event exists.
9. Check for an existing registration and confirm that the event has not reached its capacity.
10. If the checks pass, save the registration and display a success message.
11. If Report is selected, display the registrations and participant count for the chosen event.
12. For invalid input, unknown event IDs, duplicate registrations or full events, display a clear validation message.
13. Repeat the menu until the user chooses Exit.
14. Display the closing message and stop the program.

1.4.3 Data design and assumptions

The prototype stores event and registration data in memory. The main design decisions are:

- Events are maintained by an EventManager object and each event is assigned a unique event ID.
- Each registration records the student details and the event ID selected by the student.
- Event capacity must be greater than zero; registration is accepted only while places remain.
- Duplicate registration for the same event is rejected to avoid repeated records.
- The prototype is a console application and data is not retained after the program ends.

1.4.4 Flowchart

Fig 1.2 shows the flowchart of the algorithm. The diamond blocks represent decisions – the login check and the exit check – and the middle block groups the seven menu operations.

Fig 1.2 Flowchart of the college event management process

1.4.5 Class diagram

The class diagram shows Event, Student, Registration and EventManager. Event stores event information, Student stores student information, Registration links a student to an event, and EventManager coordinates event creation and registration.

Fig 1.3 Class diagram of the College Events Management System

Event categories may include technical, cultural, sports and academic events. All categories follow the same basic validation: capacity must be positive and registration is allowed only while places are available.

Table 1.3 Event categories and registration rules

Rule	TechnicalEvent	CulturalEvent
Event category	Technical fest, coding contest, workshop	Cultural fest, music, dance, drama
Capacity	Must be greater than zero	Must be greater than zero
Registration	Allowed until capacity is reached	Allowed until capacity is reached
Wrong Student ID attempts allowed	3, then event is locked	3, then event is locked

CHAPTER 2
PROJECT IMPLEMENTATION

2.1 OOP Concepts Used for Implementation

The College Events Management System is built around the main principles of Object-Oriented Programming. Each concept is used where it solves a real design problem, and Table 2.1 summarises where the concepts appear in the code.

Classes and objects:

The main entities are represented as classes: Event, Student, Registration and EventManager. Objects are created when events are added and when students register for events.

Encapsulation :

Event and registration fields are kept private where appropriate and accessed through public methods. This encapsulation helps protect data from unintended changes.

Inheritance

Common event and registration operations are organised into reusable classes and methods. Additional event categories can be added without redesigning the whole application.

Abstraction :

EventManager provides operations such as addEvent(), displayEvents(), searchEvent() and registerStudent(). The interface calls these methods without needing to manage the internal data structures directly.

Polymorphism :

The registration method checks the event ID, duplicate registration and remaining capacity before saving a registration. It reports a clear result to the user.

The College Events Management System is designed to simplify the planning and coordination of college events such as technical fests, cultural programmes, workshops, seminars and competitions. Manual management can cause duplicate registrations, unclear schedules and difficulty tracking participants. This project presents a structured system developed using C++ and Object-Oriented Programming concepts.

Input validation handles invalid capacity, missing event IDs, duplicate registrations and full events. These checks prevent incorrect registration records.

STL containers and static members

Events and registrations are stored in collections managed by EventManager. Each event has a unique ID, and registrations refer to that ID so records can be associated with the correct event.

Table 2.1 OOP concepts and their use in the project

OOP concept	Where it is used	Purpose
Class & object	Event, Registration, EventManager	Model event management entities
Constructor	Event and Registration	Initialise objects with valid data
Encapsulation	Private event and student fields	Protect and organise data
Inheritance	TechnicalEvent and CulturalEvent derive from Event	Reuse common event features
Abstraction	Common Event interface	Hide category-specific details
Polymorphism	Category-specific display methods	Support different event categories
Exception handling	Validation and error messages	Handle invalid entries safely
STL containers	Vector of events and registrations	Store collections of records
static / const	IDs and fixed labels	Maintain consistent values

2.2 Code

The system represents events and registrations as objects. An event stores its title, category, date, venue and capacity; a registration stores the student details and selected event ID.

2.2.1 Header files, helper functions and main function

```
#include <iostream>
#include <iomanip>
#include <sstream>
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>
#include <limits>

using namespace std;

int toInt(string s) {
    return atoi(s.c str());
}

string toStr(int n) {
    ostringstream os;
    os << n;
    return os.str();
}

vector<string> splitStr(const string& s, char delim) {
    vector<string> parts;
    stringstream ss(s);
    string item;
    while (getline(ss, item, delim))
        parts.push back(item);
    if (!s.empty() && s[s.size() - 1] == delim)
        parts.push back("");
    return parts;
}

// printMenu() and main() are placed at the end of CollegeEventManagementSystem.cpp
void printMenu() {
    cout << "\n== COLLEGE EVENT MANAGEMENT SYSTEM ==\n"
         << "1.Add Student 2.Add Coordinator 3.Add Venue 4.Create Event\n"
         << "5.Allocate Venue 6.Register Student 7.Mark Attendance\n"
         << "8.View Students 9.View Venues 10.View Events\n"
         << "11.Event Report 12.Save Data 13.Exit\nChoice: ";
}

int main() {
    EventManagementSystem ems;
    ems.loadData();
    int choice;
    do {
        printMenu();
        cin >> choice;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric limits<streamsize>::max(), '\n');
            cout << "Invalid input.\n";
            continue;
        }
        switch (choice) {
            case 1:  ems.addStudent(); break;
            case 2:  ems.addCoordinator(); break;
            case 3:  ems.addVenue(); break;
            case 4:  ems.addEvent(); break;
            case 5:  ems.allocateVenueToEvent(); break;
            case 6:  ems.registerStudentForEvent(); break;
            case 7:  ems.markAttendance(); break;
            case 8:  ems.displayStudents(); break;
            case 9:  ems.displayVenues(); break;
            case 10: ems.displayEvents(); break;
            case 11: ems.generateEventReport(); break;
            case 12: ems.saveData(); break;
            case 13: ems.saveData(); cout << "Exiting...\n"; break;
```

```
            default: cout << "Invalid choice.\n";
        }
    } while (choice != 13);
    return 0;
}
```

2.2.2 Person class – abstract base class

```
class Person {
protected:
    int id;
    string name;
public:
    Person(int id, string name) : id(id), name(name) {}
    virtual ~Person() {}
    int getId() const { return id; }
    string getName() const { return name; }
    virtual void display() const = 0;
    virtual string serialize() const = 0;
};
```

2.2.3 Student class

```
class Student : public Person {
    string department;
    vector<int> registeredEvents;
public:
    Student(int id, string name, string dept) : Person(id, name), department(dept) {}
    void registerForEvent(int eventId) { registeredEvents.push back(eventId); }
    bool isRegisteredFor(int eventId) const {
        return find(registeredEvents.begin(), registeredEvents.end(), eventId)
               != registeredEvents.end();
    }
    void display() const {
        cout << left << setw(8) << id << setw(20) << name << setw(15) << department
             << "Events: " << registeredEvents.size() << endl;
    }
    string serialize() const {
        stringstream ss;
        ss << id << "|" << name << "|" << department << "|";
        for (size t i = 0; i < registeredEvents.size(); i++) {
            ss << registeredEvents[i];
            if (i + 1 != registeredEvents.size()) ss << ",";
        }
        return ss.str();
    }
};
```

2.2.4 Coordinator class

```
class Coordinator : public Person {
    string department;
public:
    Coordinator(int id, string name, string dept) : Person(id, name), department(dept) {}
    void display() const {
        cout << left << setw(8) << id << setw(20) << name << department << endl;
    }
    string serialize() const {
        return toStr(id) + "|" + name + "|" + department;
    }
};
```

2.2.5 Venue class

```
class Venue {
    int id;
    string name;
    int capacity;
    bool allocated;
public:
    Venue(int id, string name, int capacity)
        : id(id), name(name), capacity(capacity), allocated(false) {}
    int getId() const { return id; }
    bool isAllocated() const { return allocated; }
    void allocate() { allocated = true; }
    void display() const {
        cout << left << setw(8) << id << setw(20) << name << setw(10) << capacity
             << (allocated ? "Allocated" : "Available") << endl;
    }
    string serialize() const {
        return toStr(id) + "|" + name + "|" + toStr(capacity) + "|" + toStr(allocated ? 1 : 0);
    }
};
```

2.2.6 Event class

```
class Event {
    int id;
    int venueId;
    int maxParticipants;
    string name;
    string date;
    string coordinatorName;
    vector<int> registeredStudents;
    vector<int> presentStudents;
public:
    Event(int id, string name, string date, string coordName, int maxP)
        : id(id), venueId(-1), maxParticipants(maxP),
          name(name), date(date), coordinatorName(coordName) {}
    int getId() const { return id; }
    string getName() const { return name; }
    string getDate() const { return date; }
    string getCoordinatorName() const { return coordinatorName; }
    void setVenueId(int v) { venueId = v; }
    int getMaxParticipants() const { return maxParticipants; }
    int getRegisteredCount() const { return (int)registeredStudents.size(); }
    vector<int> getRegisteredStudents() const { return registeredStudents; }
    vector<int> getPresentStudents() const { return presentStudents; }

    bool addRegistration(int studentId) {
        if ((int)registeredStudents.size() == maxParticipants) return false;
        if (find(registeredStudents.begin(), registeredStudents.end(), studentId)
            != registeredStudents.end()) return false;
        registeredStudents.push back(studentId);
        return true;
    }
    bool markPresent(int studentId) {
        if (find(registeredStudents.begin(), registeredStudents.end(), studentId)
            == registeredStudents.end()) return false;
        if (find(presentStudents.begin(), presentStudents.end(), studentId)
            != presentStudents.end()) return false;
        presentStudents.push_back(studentId);
        return true;
    }
```

```
    void display() const {
        string venue = (venueId == -1) ? "--" : toStr(venueId);
        cout << left << setw(6) << id << setw(20) << name << setw(12) << date
```

```
             << setw(18) << coordinatorName << setw(10) << venue
             << registeredStudents.size() << "/" << maxParticipants << endl;
    }
    string serialize() const {
        stringstream ss;
        ss << id << "|" << name << "|" << date << "|" << coordinatorName << "|"
           << maxParticipants << "|" << venueId << "|";
        for (size t i = 0; i < registeredStudents.size(); i++) {
            ss << registeredStudents[i];
            if (i + 1 != registeredStudents.size()) ss << ",";
        }
        ss << "|";
        for (size_t i = 0; i < presentStudents.size(); i++) {
            ss << presentStudents[i];
            if (i + 1 != presentStudents.size()) ss << ",";
        }
        return ss.str();
    }
};
```

2.2.7 EventManagementSystem class

```
class EventManagementSystem {
    vector<Student> students;
    vector<Coordinator> coordinators;
    vector<Venue> venues;
    vector<Event> events;
    int nextStudentId, nextCoordinatorId, nextVenueId, nextEventId;

    Student* findStudent(int id) {
        for (size_t i = 0; i < students.size(); i++)
            if (students[i].getId() == id) return &students[i];
        return nullptr;
    }
    Event* findEvent(int id) {
        for (size t i = 0; i < events.size(); i++)
            if (events[i].getId() == id) return &events[i];
        return nullptr;
    }
    Venue* findVenue(int id) {
        for (size_t i = 0; i < venues.size(); i++)
            if (venues[i].getId() == id) return &venues[i];
        return nullptr;
    }
public:
    EventManagementSystem()
        : nextStudentId(1), nextCoordinatorId(1), nextVenueId(1), nextEventId(1) {}

    void addStudent() {
        string name, dept;
        cout << "Enter student name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter department: ";
        getline(cin, dept);
        int id = nextStudentId++;
        students.push back(Student(id, name, dept));
        cout << "Student added with ID: " << id << "\n";
    }
    void addCoordinator() {
        string name, dept;
        cout << "Enter coordinator name: ";
```

```
        cin.ignore();
        getline(cin, name);
        cout << "Enter department: ";
        getline(cin, dept);
        int id = nextCoordinatorId++;
        coordinators.push_back(Coordinator(id, name, dept));
        cout << "Coordinator added with ID: " << id << "\n";
    }
    void addVenue() {
        string name;
        int capacity;
        cout << "Enter venue name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter capacity: ";
        cin >> capacity;
        int id = nextVenueId++;
        venues.push back(Venue(id, name, capacity));
        cout << "Venue added with ID: " << id << "\n";
    }
    void addEvent() {
        string name, date, coordName;
        int maxP;
        cout << "Enter event name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter date (DD-MM-YYYY): ";
        getline(cin, date);
        cout << "Enter coordinator name: ";
        getline(cin, coordName);
        cout << "Enter max participants: ";
        cin >> maxP;
        int id = nextEventId++;
        events.push_back(Event(id, name, date, coordName, maxP));
        cout << "Event created with ID: " << id << "\n";
    }
    void allocateVenueToEvent() {
        if (events.empty() || venues.empty()) {
            cout << "Need an event and a venue.\n";
            return;
        }
        int eid, vid;
        displayEvents();
        cout << "Enter Event ID: ";
        cin >> eid;
        Event* e = findEvent(eid);
        if (e == nullptr) { cout << "Event not found.\n"; return; }
        displayVenues();
        cout << "Enter Venue ID: ";
        cin >> vid;
        Venue* v = findVenue(vid);
        if (v == nullptr) { cout << "Venue not found.\n"; return; }
        if (v->isAllocated()) { cout << "Venue already allocated.\n"; return; }
        v->allocate();
        e->setVenueId(vid);
        cout << "Venue allocated to event.\n";
    }
    void registerStudentForEvent() {
        if (students.empty() || events.empty()) {
            cout << "Need a student and an event.\n";
            return;
        }
        int sid, eid;
        displayStudents();
        cout << "Enter Student ID: ";
        cin >> sid;
        Student* s = findStudent(sid);
        if (s == nullptr) { cout << "Student not found.\n"; return; }
```

```
        displayEvents();
        cout << "Enter Event ID: ";
        cin >> eid;
        Event* e = findEvent(eid);
        if (e == nullptr) { cout << "Event not found.\n"; return; }
        if (s->isRegisteredFor(eid)) { cout << "Already registered.\n"; return; }
        if (e->addRegistration(sid)) {
            s->registerForEvent(eid);
            cout << "Registered successfully.\n";
        } else {
            cout << "Registration failed: event full.\n";
        }
    }
    void markAttendance() {
        if (events.empty()) { cout << "No events.\n"; return; }
        int eid, sid;
        displayEvents();
        cout << "Enter Event ID: ";
        cin >> eid;
        Event* e = findEvent(eid);
        if (e == nullptr) { cout << "Event not found.\n"; return; }
        cout << "Enter Student ID to mark present: ";
        cin >> sid;
        if (e->markPresent(sid))
            cout << "Attendance marked.\n";
        else
            cout << "Failed: not registered or already marked.\n";
    }
    void displayStudents() const {
        if (students.empty()) { cout << "No students yet.\n"; return; }
        for (size_t i = 0; i < students.size(); i++) {
            const Person& p = students[i];
            p.display();                // polymorphic call
        }
    }
    void displayVenues() const {
        if (venues.empty()) { cout << "No venues yet.\n"; return; }
        for (size_t i = 0; i < venues.size(); i++) venues[i].display();
    }
    void displayEvents() const {
        if (events.empty()) { cout << "No events yet.\n"; return; }
        for (size_t i = 0; i < events.size(); i++) events[i].display();
    }
    void generateEventReport() const {
        if (events.empty()) { cout << "No events.\n"; return; }
        displayEvents();
        int eid;
        cout << "Enter Event ID for report: ";
        cin >> eid;
        const Event* e = nullptr;
        for (size_t i = 0; i < events.size(); i++)
            if (events[i].getId() == eid) { e = &events[i]; break; }
        if (e == nullptr) { cout << "Event not found.\n"; return; }
        cout << "\n== REPORT: " << e->getName() << " ==\n";
        cout << "Date: " << e->getDate() << " | Coordinator: " << e->getCoordinatorName() << "\n";
        cout << "Registered: " << e->getRegisteredCount() << "/" << e->getMaxParticipants() << "\n";
        vector<int> reg = e->getRegisteredStudents();
        vector<int> present = e->getPresentStudents();
        for (size_t r = 0; r < reg.size(); r++) {
            for (size_t s = 0; s < students.size(); s++) {
                if (students[s].getId() == reg[r]) {
                    bool here = find(present.begin(), present.end(), reg[r]) != present.end();
                    cout << students[s].getId() << " " << students[s].getName()
                         << " - " << (here ? "Present" : "Absent") << "\n";
                }
            }
        }
    }
```

```
    void saveData() const {
        ofstream fs("students.txt");
        for (size_t i = 0; i < students.size(); i++) fs << students[i].serialize() << "\n";
        ofstream fc("coordinators.txt");
        for (size t i = 0; i < coordinators.size(); i++) fc << coordinators[i].serialize() << "\n";
        ofstream fv("venues.txt");
        for (size t i = 0; i < venues.size(); i++) fv << venues[i].serialize() << "\n";
        ofstream fe("events.txt");
        for (size t i = 0; i < events.size(); i++) fe << events[i].serialize() << "\n";
        cout << "Data saved.\n";
    }
    void loadData() {
        string line;
        ifstream fs("students.txt");
        while (getline(fs, line)) {
            if (line.empty()) continue;
            vector<string> p = splitStr(line, '|');
            if (p.size() < 3) continue;
            Student s(toInt(p[0]), p[1], p[2]);
            if (p.size() > 3) {
                vector<string> ev = splitStr(p[3], ',');
                for (size_t i = 0; i < ev.size(); i++)
                    if (!ev[i].empty()) s.registerForEvent(toInt(ev[i]));
            }
            students.push_back(s);
            if (s.getId() >= nextStudentId) nextStudentId = s.getId() + 1;
        }
        ifstream fc("coordinators.txt");
        while (getline(fc, line)) {
            if (line.empty()) continue;
            vector<string> p = splitStr(line, '|');
            if (p.size() < 3) continue;
            Coordinator c(toInt(p[0]), p[1], p[2]);
            coordinators.push_back(c);
            if (c.getId() >= nextCoordinatorId) nextCoordinatorId = c.getId() + 1;
        }
        ifstream fv("venues.txt");
        while (getline(fv, line)) {
            if (line.empty()) continue;
            vector<string> p = splitStr(line, '|');
            if (p.size() < 3) continue;
            Venue v(toInt(p[0]), p[1], toInt(p[2]));
            if (p.size() > 3 && p[3] == "1") v.allocate();
            venues.push back(v);
            if (v.getId() >= nextVenueId) nextVenueId = v.getId() + 1;
        }
        ifstream fe("events.txt");
        while (getline(fe, line)) {
            if (line.empty()) continue;
            vector<string> p = splitStr(line, '|');
            if (p.size() < 6) continue;
            Event e(toInt(p[0]), p[1], p[2], p[3], toInt(p[4]));
            e.setVenueId(toInt(p[5]));
            if (p.size() > 6) {
                vector<string> r = splitStr(p[6], ',');
                for (size_t i = 0; i < r.size(); i++)
                    if (!r[i].empty()) e.addRegistration(toInt(r[i]));
            }
            if (p.size() > 7) {
                vector<string> pr = splitStr(p[7], ',');
                for (size t i = 0; i < pr.size(); i++)
                    if (!pr[i].empty()) e.markPresent(toInt(pr[i]));
            }
            events.push_back(e);
            if (e.getId() >= nextEventId) nextEventId = e.getId() + 1;
        }
```

2.3 Output Screens

The following output examples represent the main workflows of the College Events Management System, including event creation, event listing, student registration, search and validation messages.

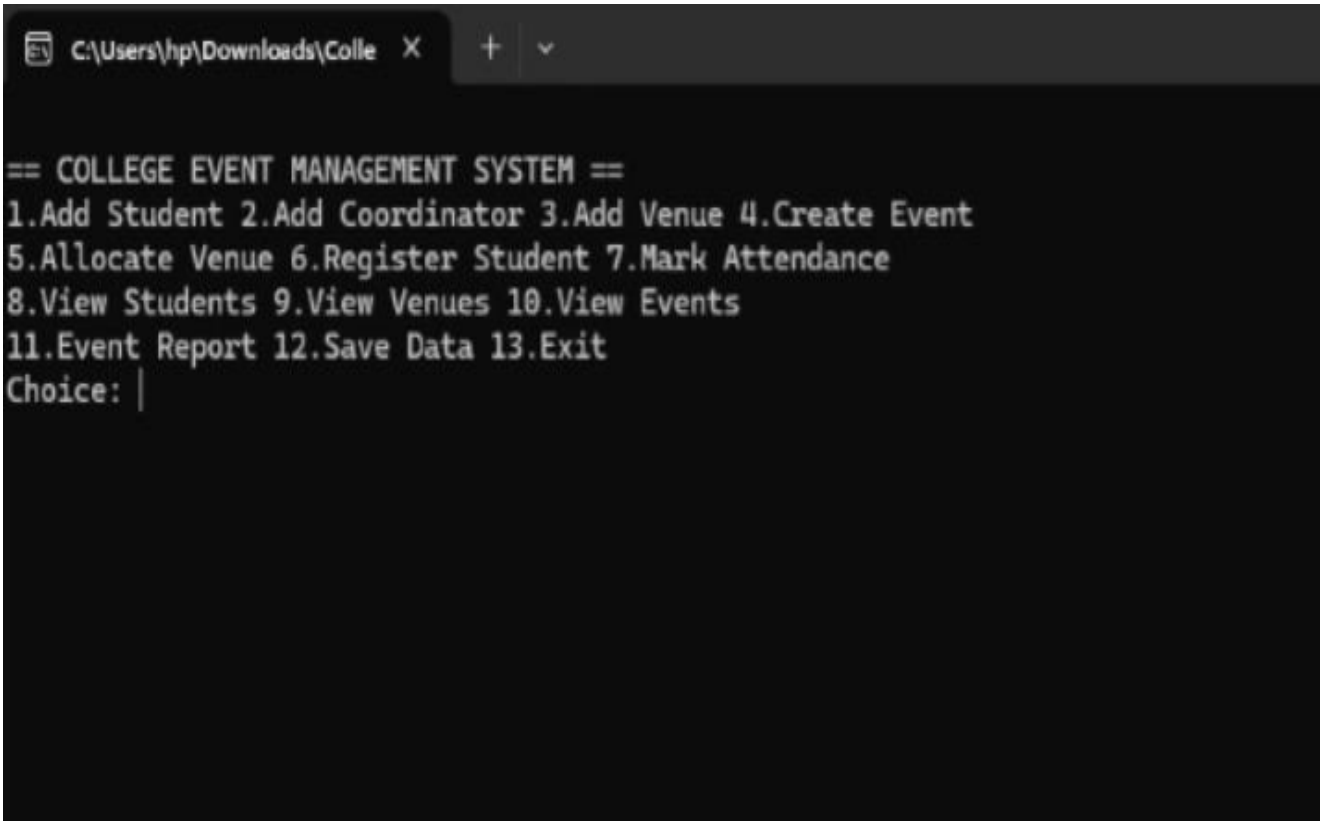


Fig 2.1 College Events Management System main menu

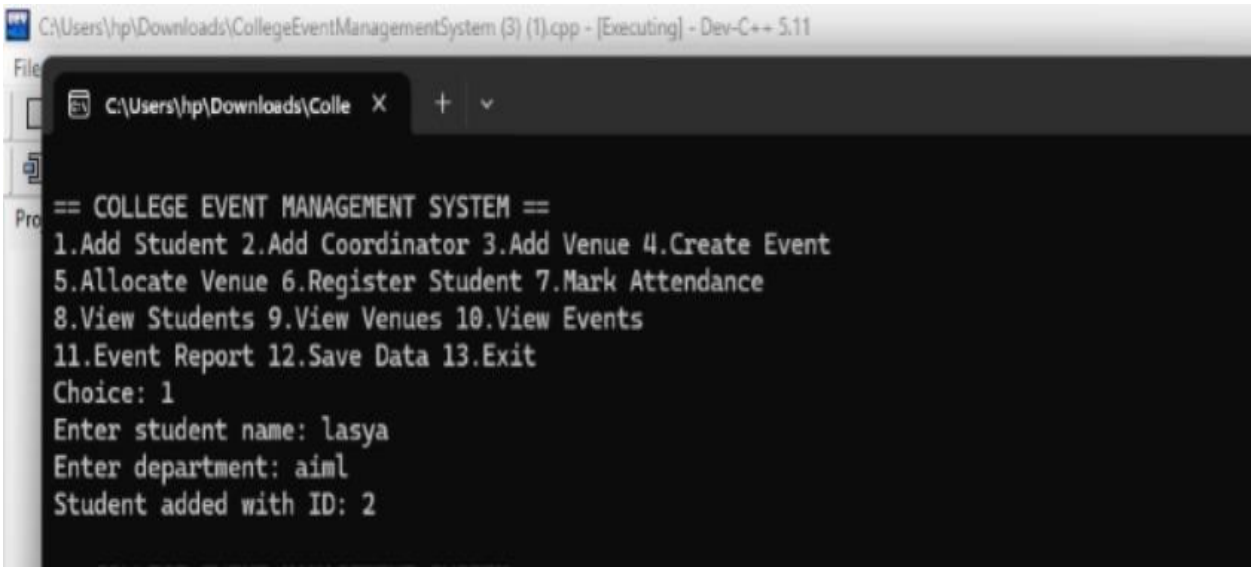


Fig 2.2 displays available events and demonstrates a student registering for an event using the event ID and student details.

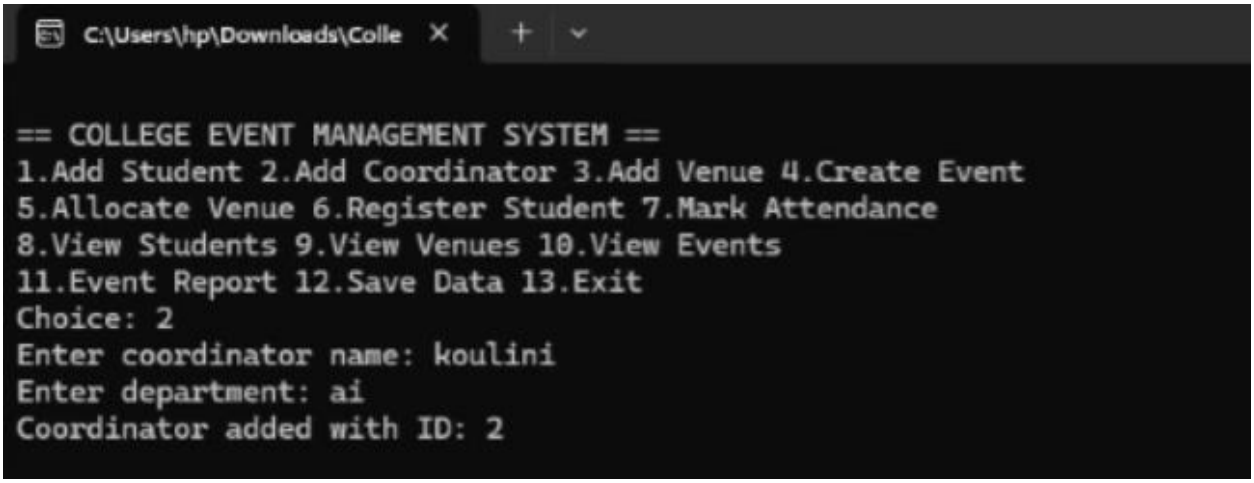


Fig 2.3 Adding coordinator

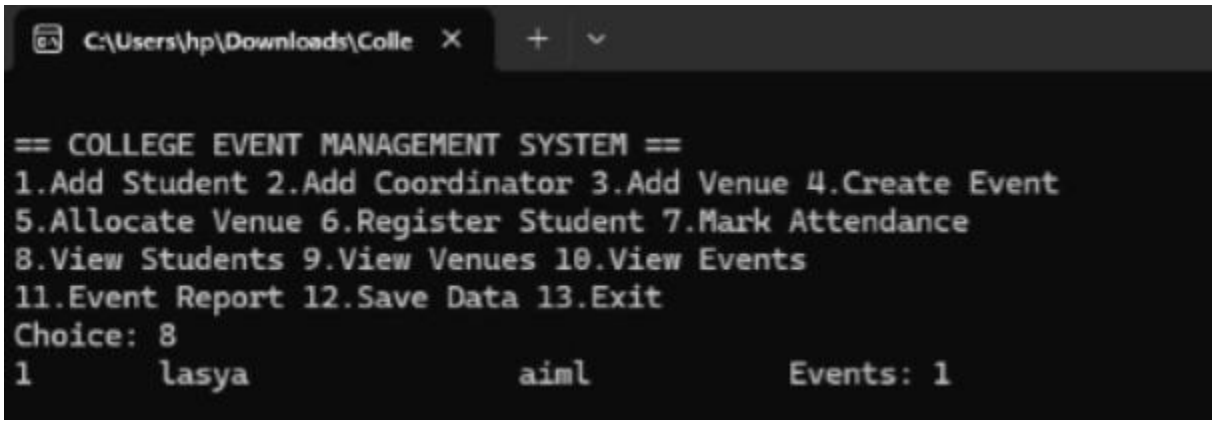


Fig 2.4 List of registered students

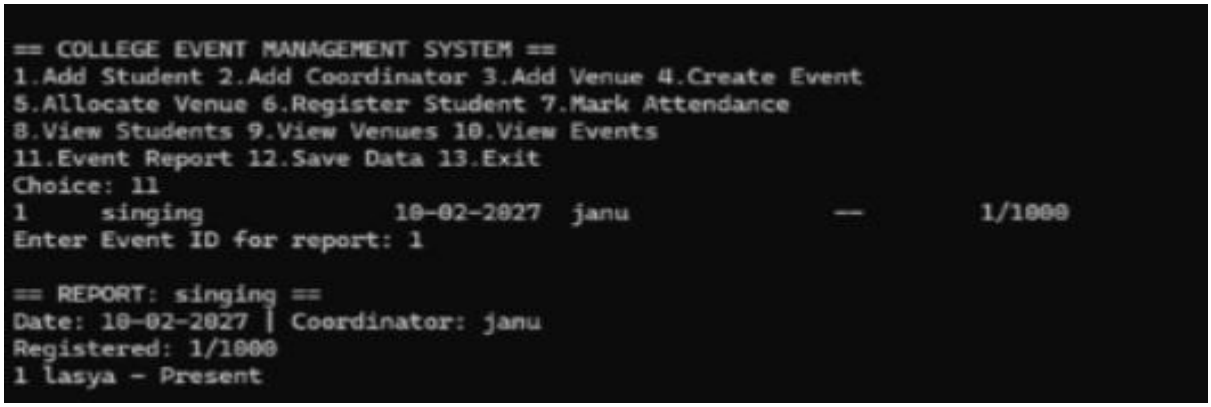


Fig 2.5 Event report

.

Table 2.2 Test cases and results

ID	Scenario	Input	Expected result	Result
TC1	Add student	Option 1, Asha, CSE	Student added with ID: 1	Pass
TC2	Add coordinator	Option 2, Dr. Rao, CSE	Coordinator added with ID: 1	Pass
TC3	Add venue	Option 3, Seminar Hall, 100	Venue added with ID: 1	Pass
TC4	Create event	Option 4, Hackathon, 2 seats	Event created with ID: 1	Pass
TC5	Allocate venue	Option 5, event 1, venue 1	Venue allocated to event	Pass
TC6	Venue already allocated	Option 5, event 1, venue 1	Venue already allocated	Pass
TC7	Register student	Option 6, student 1, event 1	Registered successfully	Pass
TC8	Duplicate registration	Option 6, student 1, event 1	Already registered	Pass
TC9	Event full	Option 6, student 3, event 1	Registration failed: event full	Pass
TC10	Mark attendance	Option 7, event 1, student 1	Attendance marked	Pass

ID	Scenario	Input	Expected result	Result
TC11	Student not registered	Option 7, event 1, student 3	Failed: not registered or already marked	Pass
TC12	Invalid input / option	abc, then 20	Invalid input / Invalid choice	Pass
TC13	Save and reload data	Option 13, restart, option 8	Saved records are loaded again	Pass

Result analysis:

All thirteen test cases passed. The results confirm that students, coordinators, venues and events are created with unique IDs, that a venue can be allocated to only one event, that registration respects the participant limit and rejects duplicates, that attendance is accepted only for registered students, and that every invalid input is rejected with a message without stopping the program. The saved text files restore all records when the program is started again.

CHAPTER 3
CONCLUSION & FUTURE SCOPE

3.1 Conclusion

The College Events Management System provides an organised way to create college events, display event details and register students. It reduces reliance on paper-based records and helps organisers maintain participant information in a consistent format.

The project demonstrates how object-oriented programming can be used in a practical application. Event, Student and Registration classes separate responsibilities, while validation rules help maintain reliable registration records.

Key outcomes of the project:

- A structured system for creating and displaying college events.
- Student registration linked to a selected event and its available capacity.
- Validation for invalid event details, duplicate registrations and full events.
- Documentation of the requirements, algorithm, architecture, class design, source code, output screens and test cases.

The current prototype stores data in memory, so records are not retained after the program closes. It uses a console interface and does not yet provide simultaneous online access for students and organisers. These are opportunities for future development.

3.2 Future Scope

The modular design allows several improvements to be added in future versions:

- A graphical or web interface can make event creation and registration easier for students and organisers.
- Database integration with MySQL or PostgreSQL can preserve event details and registration records permanently.
- Login accounts and organiser roles can control who is allowed to create, edit or cancel events.
- Additional features can include attendance tracking, certificates, feedback forms, reminders and waiting lists.

- Reports and dashboards can show registration counts, available seats and event participation.
- A mobile-friendly website can allow students to browse events and register remotely.
- Automated tests and activity logs can improve reliability and help organisers review changes.

REFERENCES

[1] Bjarne Stroustrup, A Tour of C++, Addison-Wesley.

[2] Herbert Schildt, C++: The Complete Reference, McGraw-Hill Education.

[3] cppreference.com, C++ Reference Documentation, https://en.cppreference.com/.

[4] Ian Sommerville, Software Engineering, Pearson.

[5] Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.